

Session 10 : Introduction à la Physique des Rotations

Objectifs de la session

- Passer de la **particule** (point sans dimension) au **corps rigide** (objet qui pivote).
- Comprendre le **couple**, le **moment d'inertie** et la **vitesse angulaire**.
- Relier les équations linéaires et angulaires — elles sont **analogues**.
- Modéliser la **friction de glissement vs roulement** — le cas de la boule de billard.
- Implémenter un corps rigide qui se déplace **et** tourne, avec intégration par quaternion.

L'objectif du jour

Depuis le début du cours, on traite les objets comme des **points** — une position, une vitesse, une masse. Mais un vrai objet ne fait pas que se déplacer : il **pivote** autour de son centre de masse. Une boule de billard roule, un cube tombe en basculant, un personnage s'effondre en pivotant. Cette session introduit les outils pour modéliser la rotation — et on le fait avec un cas concret : **la boule de billard**.

Partie 1 : Du Point au Corps Rigide

Particule vs corps rigide

- **Particule** (sessions 1-9) : un point avec une masse m , une position $\vec{r}$, une vitesse $\vec{v}$. Pas d'orientation, pas de rotation. Toute la masse est concentrée en un point.
- **Corps rigide** : un objet avec une **étendue spatiale**. Il a un **centre de masse** (G), une **orientation** (rotation), et une **répartition de masse** qui détermine comment il résiste à la rotation.

Un corps rigide peut faire **deux choses simultanément** :

1. **Translater** — son centre de masse se déplace (comme une particule).
2. **Pivoter** — il tourne autour de son centre de masse.

L'hypothèse rigide

On dit **rigide** parce que les points internes de l'objet ne bougent pas les uns par rapport aux autres — la forme ne se déforme pas. C'est une approximation : un vrai objet se déforme légèrement sous les forces. Mais pour une boule de billard, un cube de bois, un personnage en rigid body — c'est excellent.

Partie 2 : Le Dictionnaire Linéaire → Angulaire

Tout ce que vous savez s'applique

La physique de la rotation est **exactement parallèle** à la physique de la translation. Chaque concept linéaire a un équivalent angulaire. Si vous comprenez $\vec{F} = m\vec{a}$, vous comprenez $\tau = I\alpha$.

Concept	Linéaire	Angulaire
Position	$\vec{r}$ (m)	Angle θ (rad) / Quaternion q
Vitesse	$\vec{v} = d \cdot \vec{r} / dt$ (m/s)	$\vec{\omega} = d \cdot \vec{\theta} / dt$ (rad/s)
Accélération	$\vec{a}$ (m/s ²)	$\vec{\alpha}$ (rad/s ²)
Résistance	Masse m (kg)	Moment d'inertie I (kg·m ²)
Cause du mouvement	Force $\vec{F}$ (N)	Couple $\vec{\tau}$ (N·m)
Loi du mouvement	$\vec{F} = m\vec{a}$	$\vec{\tau} = I\vec{\alpha}$
Quantité de mouvement	$\vec{p} = m\vec{v}$	Moment angulaire $\vec{L} = I\vec{\omega}$
Énergie cinétique	$1/2mv^2$	$1/2I\omega^2$

Table 1: Analogie entre mouvement linéaire et rotation. Chaque équation linéaire a sa contrepartie angulaire — il suffit de remplacer les variables.

💡 Mémoriser le tableau

La masse m mesure la **résistance à la translation** — plus m est grand, plus il est difficile d'accélérer. Le moment d'inertie I mesure la **résistance à la rotation** — plus I est grand, plus il est difficile de faire tourner. La force $\vec{F}$ cause la translation, le couple $\vec{\tau}$ cause la rotation. **Tout le reste suit.**

Partie 3 : Le Couple (Torque)

Définition

Le couple est l'équivalent angulaire de la force. Mais contrairement à une force, le couple dépend de **deux** choses :

1. La force appliquée $\vec{F}$.
2. Le **point d'application** — ou plus précisément, le **bras de levier** $\vec{r}$, la distance entre le centre de masse et le point où la force est appliquée.

En 3D, le couple est un produit vectoriel :

$$\vec{\tau} = \vec{r} \times \vec{F}$$

En 2D, il se simplifie à un scalaire :

$$\tau = r_x F_y - r_y F_x$$

💡 Pourquoi le point d'application compte

Pousser une porte près de la charnière ne produit presque rien. Pousser la même porte avec la même force **au bout de la poignée** la fait pivoter facilement. La force est la même, mais le **bras de levier** est différent — et le couple est proportionnel au bras de levier.

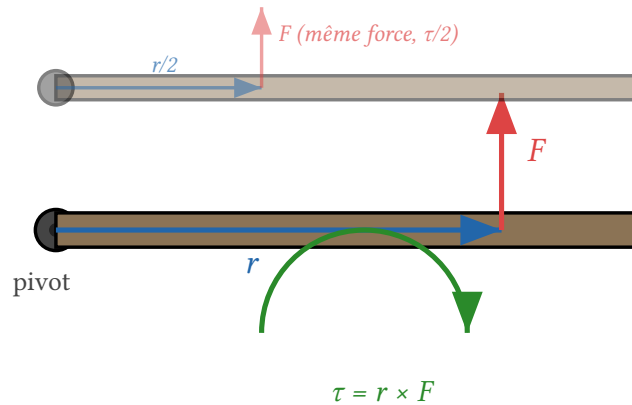


Figure 1: Le couple dépend du bras de levier. En haut : force au bout de la porte (r grand, τ grand). En bas : même force au milieu ($r/2$, τ divisé par 2). Pousser près du pivot produit peu de rotation.

Exemples intuitifs

Exemple 1 : La porte

Une porte de 0.8 “m” de large. On pousse avec 10 “N” :

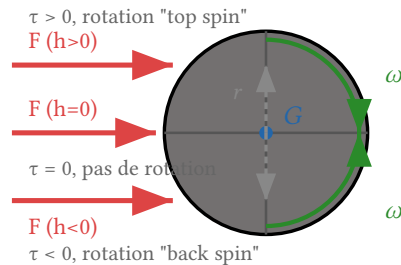
- **Au bout** ($r = 0.8$) : $\tau = 0.8 \times 10 = 8$ “N”·“m”. La porte s’ouvre facilement.
- **Près de la charnière** ($r = 0.1$) : $\tau = 0.1 \times 10 = 1$ “N”·“m”. La porte bouge à peine.
- **Sur la charnière** ($r = 0$) : $\tau = 0$. La porte ne tourne pas, peu importe la force.

Exemple 2 : La clé

Serrer un boulon avec une clé de 0.2 “m” et une force de 50 “N” perpendiculaire : $\tau = 0.2 \times 50 = 10$ “N”·“m”. Si on ajoute un prolongateur de 0.4 “m” : $\tau = 0.4 \times 50 = 20$ “N”·“m” — le double. C’est pourquoi les mécaniciens utilisent des clés longues.

Exemple 3 : La boule de billard

On frappe la boule avec une queue. Si on frappe **au centre** ($h = 0$), le bras de levier est nul — pas de couple, la boule glisse sans tourner. Si on frappe **haut** ($h > 0$), on crée un couple — la boule tourne. C’est le **top spin** (rotation vers l’avant). Si on frappe **bas** ($h < 0$), c’est le **back spin** (rotation vers l’arrière).



$$\tau = \mathbf{r} \times \mathbf{F} \rightarrow \omega = \tau / I$$

Figure 2: Coup de queue centré vs décentré sur une boule de billard. La hauteur d'impact h détermine le bras de levier r et donc le couple $\tau = r \times F$. Un coup centré ($h = 0$) ne produit pas de rotation ; un coup haut produit du top spin ; un coup bas produit du back spin.

Partie 4 : Le Moment d'Inertie

Définition

Le moment d'inertie I représente la **résistance d'un objet à la rotation** — exactement comme la masse résiste à la translation. Il dépend de la **répartition de la masse** par rapport à l'axe de rotation :

$$I = \sum_i m_i r_i^2$$

où m_i est la masse de chaque point et r_i sa distance à l'axe. Plus la masse est **loin** de l'axe, plus I est grand — et plus il est difficile de faire tourner.

L'intuition du patineur

Un patineur qui tourne sur lui-même rapproche ses bras de son corps $\rightarrow$ il tourne **plus vite**. Pourquoi ? Parce que rapprocher les bras diminue r_i pour les masses des bras, donc diminue I . Or le moment angulaire $L = I\omega$ se **consERVE** (pas de couple externe). Si I diminue, ω augmente — le patineur accélère. C'est la conservation du moment angulaire, l'équivalent de la conservation de quantité de mouvement.

Formules pour les formes courantes

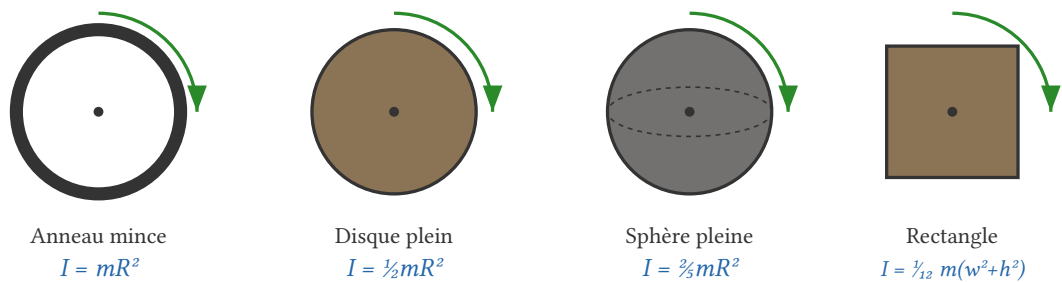


Figure 3: Moments d'inertie pour des formes courantes autour de leur centre. Notez comment la masse répartie **loin** de l'axe (anneau) donne un I plus grand que la masse concentrée au centre (disque).

Cas courants (autour du centre de masse)

- **Anneau mince** (masse sur le bord) : $I = mR^2$ — toute la masse est à distance R .
- **Disque plein / cylindre** : $I = \frac{1}{2}mR^2$ — la masse est répartie de 0 à R .
- **Sphère pleine** : $I = \frac{2}{5}mR^2$ — la masse est répartie en 3D.
- **Sphère creuse** : $I = \frac{2}{3}mR^2$ — masse concentrée sur la surface.
- **Rectangle** (autour du centre, dans le plan) : $I = \frac{1}{12}m(w^2 + h^2)$.
- **Tige fine** (autour du centre) : $I = \frac{1}{12}mL^2$.
- **Tige fine** (autour d'une extrémité) : $I = \frac{1}{3}mL^2$.

💡 La boule de billard

Une boule de billard est une **sphère pleine** de masse $m = 0.2$ "kg" et rayon $R = 0.5$ (unités de la démo) :

$$I = \frac{2}{5}mR^2 = \frac{2}{5} \times 0.2 \times 0.25 = 0.02 \text{ kg} \cdot \text{m}^2$$

C'est cette valeur que l'on utilise dans la démo [billiard.html](#).

Le théorème des axes parallèles

Théorème de Huygens-Steiner

Si on connaît I_{cm} autour du centre de masse, on peut calculer I autour de **n'importe quel axe parallèle** à une distance d :

$$I = I_{\text{cm}} + md^2$$

Exemple : une tige de masse m et longueur L a $I_{\text{cm}} = \frac{1}{12}mL^2$ autour de son centre. Autour d'une extrémité ($d = \frac{L}{2}$) :

$$I = \frac{1}{12}mL^2 + m\left(\frac{L}{2}\right)^2 = \frac{1}{12}mL^2 + \frac{1}{4}mL^2 = \frac{1}{3}mL^2$$

On retrouve la formule de la tige autour d'une extrémité. Le théorème **marche**.

Partie 5 : Vitesse d'un Point sur le Corps

Le point crucial

Quand un corps rigide se déplace **et** tourne, la vitesse de **chaque point** sur le corps est différente. Le centre de masse a une vitesse $\vec{v}_G$. Mais un point sur le bord a une vitesse **différente** — il subit la translation **plus** la rotation.

Vitesse d'un point P

Si un corps se déplace à $\vec{v}_G$ et tourne à $\vec{\omega}$, la vitesse d'un point P à la position $\vec{r}$ (relative au centre de masse) est :

$$\vec{v}_P = \vec{v}_G + (\vec{\omega} \times \vec{r})$$

En 2D, avec $\vec{r} = (r_x, r_y)$ et $\vec{\omega} = \omega$ (scalaire, perpendiculaire au plan) :

$$v_{P.x} = v_{G.x} - \omega \cdot r_y$$

$$v_{P.y} = v_{G.y} + \omega \cdot r_x$$

$$v_P = v_G + (\omega \times r)$$

Si $v_P \neq 0$: la boule glisse (friction cinétique)

Si $v_P = 0$: la boule roule sans glisser

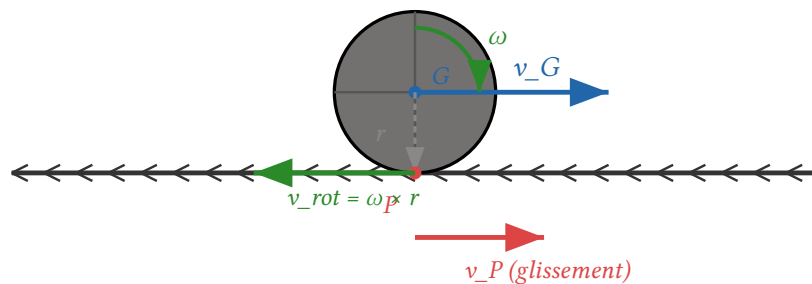


Figure 4: Vitesse du point de contact P sur une boule qui roule. Le centre G se déplace à $\vec{v}_G$ (bleu). La rotation $\vec{\omega}$ (vert) crée une vitesse $\vec{\omega} \times \vec{r}$ au point de contact (vert, vers l'arrière). La vitesse **réelle** du point de contact est $\vec{v}_P = \vec{v}_G + (\vec{\omega} \times \vec{r})$ (rouge). Si $\vec{v}_P \neq 0$, la boule **glisse**. Si $\vec{v}_P = 0$, la boule **roule sans glisser**.

Exemple : la boule qui glisse vs roule

Cas 1 : Glissement pur (pas de rotation)

On frappe la boule au centre ($h = 0$) $\rightarrow$ pas de couple $\rightarrow \omega = 0$. La boule se déplace à $\vec{v}_G$ mais ne tourne pas. Au point de contact :

$$\vec{v}_P = \vec{v}_G + (0 \times \vec{r}) = \vec{v}_G$$

Le point de contact **avance** à la même vitesse que le centre. La boule **glisse** sur le tapis — le tapis frotte.

Cas 2 : Roulement pur

La boule tourne **assez vite** pour que le point de contact soit **immobile** par rapport au sol.
Condition :

$$\vec{v}_P = \vec{v}_G + (\vec{\omega} \times \vec{r}) = 0$$

$$\vec{v}_G = -(\vec{\omega} \times \vec{r}) = \vec{\omega} \times (-\vec{r})$$

En 2D, avec $\vec{r} = (0, -R)$ (contact en bas) :

$$v_G = \omega R$$

C'est la **condition de roulement sans glissement** : $v_G = \omega R$. La boule avance exactement à la vitesse qu'il faut pour que le point de contact soit immobile.

Cas 3 : Glissement + rotation (cas général)

La boule a $\vec{v}_G$ et ω , mais $v_G \neq \omega R$. Le point de contact a une vitesse **non nulle** — c'est la **vitesse de glissement**. La friction va agir pour **réduire** cette vitesse de glissement jusqu'à atteindre le roulement pur. C'est ce qu'on modélise dans la démo.

Partie 6 : Friction — Glissement vs Roulement

! Le cœur de la démo billard

Quand la boule glisse ($v_P \neq 0$), le tapis exerce une **force de friction cinétique** qui s'oppose au glissement. Cette force fait **deux choses** simultanément :

1. Elle **ralentit** la translation (réduit v_G).
2. Elle **accélère** la rotation (augmente ω) — car la friction appliquée au point de contact crée un couple.

La friction **converge** le système vers le roulement pur ($v_G = \omega R$). Une fois le roulement atteint, la friction de glissement s'arrête — il ne reste que la **résistance au roulement** (beaucoup plus faible).

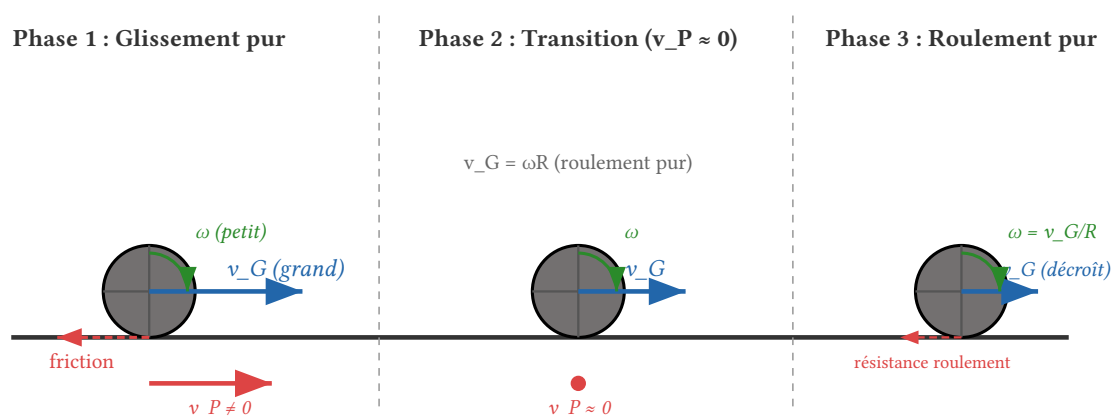


Figure 5: Les trois phases d'une boule de billard. Phase 1 : glissement pur, $v_G \gg \omega R$, friction cinétique forte (ralentit v_G , accélère ω). Phase 2 : transition, $v_G \approx \omega R$, la friction de glissement devient nulle. Phase 3 : roulement pur, seule la résistance au roulement (très faible) décélère lentement la boule.

La force de friction cinétique

Friction de glissement

Quand la boule glisse, le tapis exerce une force de friction cinétique :

$$\vec{F}_{\text{friction}} = -\mu_{\text{slide}} \cdot m \cdot g \cdot \hat{v}_P$$

où :

- μ_{slide} : coefficient de friction cinétique (≈ 0.2 pour un tapis de billard).
- $m \cdot g$: poids de la boule (force normale du sol).
- $\hat{v}_P$: direction de la vitesse de glissement (normalisée).

La force est **opposée** à la vitesse de glissement — elle tend à **annuler** $\vec{v}_P$.

Effet sur la translation

La friction décélère le centre de masse :

$$\vec{a}_G = \frac{\vec{F}_{\text{friction}}}{m} = -\mu_{\text{slide}} \cdot g \cdot \hat{v}_P$$

L'accélération est **constante** en magnitude ($\mu_{\text{slide}} \cdot g$) et opposée au glissement. La vitesse linéaire diminue.

Effet sur la rotation

La même force de friction, appliquée au point de contact, crée un **couple** :

$$\vec{\tau} = \vec{r} \times \vec{F}_{\text{friction}}$$

où $\vec{r} = (0, -R, 0)$ (du centre vers le contact). Ce couple **accélère** la rotation :

$$\vec{\alpha} = \frac{\vec{\tau}}{I}$$

Pour une sphère pleine ($I = \frac{2}{5}mR^2$) avec friction horizontale F :

$$\alpha = \frac{R \cdot F}{\frac{2}{5}mR^2} = \frac{5F}{2mR}$$

La rotation augmente — la boule se met à tourner **plus vite**.

La friction fait deux choses opposées

Paradoxalement, la **même** force de friction :

- **Ralentit** la translation (v_G diminue).
- **Accélère** la rotation (ω augmente).

C'est exactement ce qu'il faut pour converger vers $v_G = \omega R$: v_G descend, ω monte, jusqu'à se rencontrer. C'est la beauté de la physique du roulement.

La résistance au roulement

Friction de roulement

Une fois le roulement pur atteint ($v_P = 0$), il n'y a plus de glissement — la friction cinétique s'annule. Mais la boule finit quand même par s'arrêter. Pourquoi ?

La **résistance au roulement** est un effet différent : la déformation microscopique du tapis sous la boule crée une force **très faible** qui décélère la boule. On la modélise simplement :

$$\vec{v} * = (1 - \mu_{\text{roll}} \cdot dt \cdot k)$$

où μ_{roll} est très petit (≈ 0.02) et k un facteur empirique. C'est un **amortissement** simple, pas une vraie force de friction.

💡 Dans la démo [billiard.html](#)

Le code distingue les deux modes :

```
if (slideSpeed > 0.05) {  
    // Mode glissement : friction cinétique  
    // - Ralentit v_G (linéaire)  
    // - Accélère omega (couple)  
} else {  
    // Mode roulement : résistance simple  
    // - Amortit v et omega ensemble  
}
```

Le seuil 0.05 est la tolérance — en dessous, on considère que $v_P \approx 0$ et on passe en mode roulement.

Partie 7 : Intégration de la Rotation

Le problème de l'orientation

En translation, on intègre : $\vec{v} + = \vec{a} \cdot dt$, puis $\vec{r} + = \vec{v} \cdot dt$. Simple.

En rotation, c'est pareil pour la vitesse angulaire : $\vec{\omega} + = \vec{\alpha} \cdot dt$. Mais pour l'**orientation** — comment on met à jour l'angle ? En 2D, $\theta + = \omega \cdot dt$ suffit. En 3D, c'est plus subtil.

2D : angle scalaire

Rotation 2D

En 2D, la rotation est un simple angle scalaire θ :

```
angularVelocity += (torque / inertia) * dt;  
angle += angularVelocity * dt;
```

On peut utiliser `ctx.rotate(angle)` pour le rendu. Simple et suffisant pour un jeu 2D.

3D : quaternions

❗ Pourquoi pas les angles d'Euler ?

En 3D, on **pourrait** utiliser trois angles (Euler : lacet, tangage, roulis). Mais les angles d'Euler souffrent du **gimbal lock** — une perte d'un degré de liberté quand deux axes s'alignent. De plus, interpoler entre deux orientations Euler donne des rotations **non naturelles** (le chemin le plus court n'est pas pris).

Les **quaternions** résolvent ces problèmes. Un quaternion $q = (w, x, y, z)$ représente une rotation de w radians autour de l'axe (x, y, z) normalisé. Pas de gimbal lock, interpolation naturelle (slerp).

Intégration par quaternion

Pour intégrer la rotation en 3D :

1. Calculer l'axe et l'angle de rotation ce frame : $\vec{axis} = \hat{\vec{\omega}}, \angle = |\vec{\omega}| \cdot dt$.
2. Créer un quaternion de rotation : $q_{\text{delta}} = \text{setFromAxisAngle}(\vec{axis}, \angle)$.
3. Multiplier avec l'orientation actuelle : $q_{\text{new}} = q_{\text{delta}} \cdot q_{\text{old}}$.

```
const axis = angularVelocity.clone().normalize();
const angle = angularVelocity.length() * dt;
if (angle > 1e-6) {
    const dq = new THREE.Quaternion().setFromAxisAngle(axis, angle);
    quaternion.premultiply(dq); // q_new = dq * q_old
}
```

Le `premultiply` applique la rotation dans le **repère monde** — si on voulait la rotation dans le repère **local** de l'objet, on utiliserait `multiply`.

💡 Euler explicite pour la rotation

Comme pour la translation, on utilise **Euler explicite** pour la rotation :

$$\vec{\omega}_{\text{new}} = \vec{\omega} + \left(\frac{\vec{\tau}}{I} \right) \cdot dt$$

$$q_{\text{new}} = \text{quat}(\hat{\vec{\omega}}, |\vec{\omega}| \cdot dt) \cdot q_{\text{old}}$$

Les mêmes problèmes de stabilité s'appliquent — Euler dérive en énergie. Mais pour une boule de billard qui s'arrête en quelques secondes (forte dissipation), c'est parfaitement adapté. Pour un satellite en orbite pendant des heures (pas de dissipation), il faudrait un intégrateur symplectique.

Partie 8 : La Démo — Billard (billiard.html)

💡 La démo

Ouvrir `exemples/billiard.html` dans un navigateur. La démo simule une boule de billard sur un tapis, avec :

- Un coup de queue paramétrable (force + hauteur d'impact).
- La distinction glissement vs roulement.
- La friction cinétique qui ralentit v_G et accélère ω .
- La résistance au roulement qui arrête lentement la boule.

- Des flèches de visualisation : vitesse $\vec{v}_G$ (bleu) et vitesse de glissement $\vec{v}_P$ (rouge).

Anatomie du code

État physique

```
const physicsState = {
  pos: new THREE.Vector3(-4, BALL_RADIUS, 0), // position du centre de masse
  vel: new THREE.Vector3(0, 0, 0),           // v_G (vitesse linéaire)
  angVel: new THREE.Vector3(0, 0, 0),         // ω (vitesse angulaire)
  quat: new THREE.Quaternion(),              // orientation (quaternion)
  isSliding: false                           // mode : glissement vs roulement
};
```

C'est le **corps rigide** — position, vitesse linéaire, vitesse angulaire, orientation. Exactement le dictionnaire de la Partie 2.

Le coup de queue (impulsion)

```
function shootBall() {
  resetGame();
  syncFromParams();

  // 1. Impulsion linéaire :  $v_G = F / m$ 
  const direction = new THREE.Vector3(1, 0, 0);
  const linearSpeed = IMPULSE_FORCE / BALL_MASS;
  physicsState.vel.copy(direction.multiplyScalar(linearSpeed));

  // 2. Impulsion angulaire :  $\tau = r \times F \rightarrow \omega = \tau / I$ 
  //    r = (0, h, 0) est le bras de levier (hauteur d'impact)
  const r = new THREE.Vector3(0, IMPACT_HEIGHT, 0);
  const F = new THREE.Vector3(IMPULSE_FORCE, 0, 0);
  const torqueImpulse = new THREE.Vector3().crossVectors(r, F);
  physicsState.angVel.copy(torqueImpulse.divideScalar(INERTIA));
}
```

Deux choses se passent :

1. La force donne une vitesse linéaire $v = \frac{F}{m}$.
2. Le couple $\tau = r \times F$ donne une vitesse angulaire $\omega = \frac{\tau}{I}$.

Le paramètre `impactHeight` (h) contrôle le bras de levier — c'est la **hauteur** où la queue frappe la boule. À $h = 0$ (centre), pas de rotation. À $h > 0$, top spin. À $h < 0$, back spin.

Le calcul du glissement

```
// Vecteur du centre vers le point de contact : r = (0, -R, 0)
const rVector = new THREE.Vector3(0, -BALL_RADIUS, 0);

// Vitesse du point de contact :  $v_P = v_G + (\omega \times r)$ 
const rotationalVelAtPoint = new THREE.Vector3()
  .crossVectors(omega, rVector);
const slideVel = new THREE.Vector3()
  .addVectors(v, rotationalVelAtPoint);
slideVel.y = 0; // on ignore la composante verticale
```

```
const slideSpeed = slideVel.length();
```

C'est **exactement** la formule de la Partie 5 : $\vec{v}_P = \vec{v}_G + (\vec{\omega} \times \vec{r})$. Si `slideSpeed > SLIDE_THRESHOLD`, la boule glisse. Sinon, elle roule.

La friction (mode glissement)

```
if (slideSpeed > SLIDE_THRESHOLD) {
  // Force de friction : opposée à la vitesse de glissement
  const frictionDir = slideVel.clone().normalize().negate();
  const frictionMag = MU_SLIDE * BALL_MASS * GRAVITY;
  const frictionForce = frictionDir.multiplyScalar(frictionMag);

  // A. Effet linéaire : ralentit v_G
  const linearAcc = frictionForce.clone().divideScalar(BALL_MASS);
  v.addScaledVector(linearAcc, dt);

  // B. Effet angulaire :  $\tau = r \times F \rightarrow$  accélère  $\omega$ 
  const torque = new THREE.Vector3()
    .crossVectors(rVector, frictionForce);
  const angularAcc = torque.divideScalar(INERTIA);
  omega.addScaledVector(angularAcc, dt);
}
```

La friction fait **deux choses** :

1. **A.** Décélère v_G (la boule ralentit linéairement).
2. **B.** Crée un couple $\tau = r \times F$ qui **accélère** ω (la boule tourne plus vite).

C'est la convergence vers $v_G = \omega R$ — la boule passe du glissement au roulement.

La résistance (mode roulement)

```
} else {
  // Décroissance exponentielle : frame-rate independent
  const dragFactor = Math.exp(-MU_ROLL * dt);
  v.multiplyScalar(dragFactor);

  // Friction statique à basse vitesse : garantit l'arrêt
  // (l'exponentielle seule n'atteint jamais zéro)
  const speed = v.length();
  if (speed > 0) {
    const staticDrag = 0.2 * dt; // décélération constante (m/s²)
    const newSpeed = Math.max(0, speed - staticDrag);
    v.multiplyScalar(newSpeed / speed);
  }

  // Contraindre  $\omega$  à la condition de roulement :  $v_G = \omega \times r$ 
  // (évite l'oscillation glissement→roulement)
  omega.set(v.z / BALL_RADIUS, 0, -v.x / BALL_RADIUS);

  // Seuil d'arrêt
  if (v.lengthSq() < 0.02) {
    v.set(0, 0, 0);
  }
}
```

```

        omega.set(0, 0, 0);
    }
}

```

Trois choses se passent en mode roulement :

1. **Décroissance exponentielle** — la résistance au roulement ralentit v_G de façon continue ($\exp(-\mu_r \cdot dt)$). Frame-rate independent.
2. **Friction statique** — une décélération **constante** qui garantit l'arrêt (l'exponentielle seule n'atteint jamais zéro, seulement asymptotiquement).
3. **Contrainte de roulement** — on recalcule ω depuis v_G pour maintenir $v_G = \omega R$ **exactement**. Sans cela, les erreurs de virgule flottante font osciller la boule entre glissement et roulement.

L'intégration (position + orientation)

```

// Position : r += v * dt (Euler)
physicsState.pos.addScaledVector(v, dt);

// Orientation : quaternion (Partie 7)
const axis = omega.clone().normalize();
const angle = omega.length() * dt;
if (angle > 1e-6) {
    const q = new THREE.Quaternion()
        .setFromAxisAngle(axis, angle);
    physicsState.quat.premultiply(q); // q_new = dq * q_old
}

```

Translation en Euler, rotation en quaternion. Le seuil 1e-6 évite les calculs inutiles quand $\omega \approx 0$.

Collisions avec les murs

```

let hitWall = false;
if (pos.x > halfX) { pos.x = halfX; v.x *= -0.8; hitWall = true; }
if (pos.x < -halfX) { pos.x = -halfX; v.x *= -0.8; hitWall = true; }
if (pos.z > halfZ) { pos.z = halfZ; v.z *= -0.8; hitWall = true; }
if (pos.z < -halfZ) { pos.z = -halfZ; v.z *= -0.8; hitWall = true; }

// Après un rebond, recalculer ω pour maintenir la condition de roulement
// (sinon v_G et ω sont désynchronisés → re-glissement parasite)
if (hitWall) {
    omega.set(v.z / BALL_RADIUS, 0, -v.x / BALL_RADIUS);
}

```

Le rebond inverse la vitesse ($\times -0.8$: 20% de perte d'énergie), mais **sans** mettre à jour ω , la condition $v_G = \omega R$ est cassée — la boule re-entre en mode glissement au frame suivant. On resynchronise ω immédiatement après le rebond.

Expériences à essayer

Manipulations

1. **Coup centré** ($h = 0$) : la boule glisse sans tourner. Observer la flèche rouge (vitesse de glissement) — elle est égale à v_G . La friction met du temps à faire rouler la boule.

2. **Coup haut** ($h = +0.3$) : top spin. La boule tourne vers l'avant. Si $\omega R > v_G$, le point de contact va **vers l'arrière** — la friction **accélère** la boule ! (Effet de "rattrapage" du top spin.)
3. **Coup bas** ($h = -0.3$) : back spin. La boule tourne vers l'arrière. Le point de contact va **vers l'avant** — la friction **décélère** la boule plus vite. Observer la boule qui recule après s'être arrêtée (effet de "retour" du back spin).
4. **Force faible** ($F = 2$) vs **force forte** ($F = 15$) : la force change v_G mais pas ω (si h est le même). Observer comment la **durée** du glissement change — plus de glissement = plus de temps avant le roulement.
5. **Friction glissement** ($\mu_s = 0$) : la boule glisse indéfiniment sans jamais se mettre à rouler (surface de glace). À $\mu_s = 2$, elle se met à rouler presque instantanément.
6. **Friction roulement** ($\mu_r = 0$) : une fois en roulement, la boule ne s'arrête jamais (roulement parfait). À $\mu_r = 0.5$, elle s'arrête très vite.
7. **Vitesse temps** (0.1) : ralentir la simulation pour observer la transition glissement $\rightarrow$ roulement en détail. La flèche rouge (glissement) rétrécit jusqu'à disparaître, puis la boule roule.

Partie 9 : TP — Corps Rigide 2D

🔗 Objectif du TP

Implémenter un **corps rigide 2D** — un rectangle qui se déplace **et** tourne quand on clique dessus. Le principe est le même que la boule de billard, mais en 2D avec un angle scalaire au lieu d'un quaternion.

Étape 1 : Classe RigidBody

Ajouter les propriétés angulaires à un objet qui a déjà position, velocity, mass :

```
class RigidBody {
    constructor(width, height, mass) {
        // --- Linéaire (déjà vu) ---
        this.pos = new Vector2(0, 0);
        this.vel = new Vector2(0, 0);
        this.mass = mass;
        this.force = new Vector2(0, 0); // accumulateur

        // --- Angulaire (nouveau) ---
        this.angle = 0; //  $\theta$  (radians)
        this.angularVelocity = 0; //  $\omega$  (rad/s)
        this.torque = 0; //  $\tau$  (accumulateur)
        // Moment d'inertie d'un rectangle :  $I = (1/12) m (w^2 + h^2)$ 
        this.inertia = (1/12) * mass * (width*width + height*height);
    }
}
```

Étape 2 : ApplyForce(force, worldPoint)

La méthode clé

```
applyForce(force, worldPoint) {
    // 1. Force linéaire :  $F_{total} += F$ 
```

```

    this.force.x += force.x;
    this.force.y += force.y;

    // 2. Couple :  $\tau = r_x * F_y - r_y * F_x$ 
    const r = {
      x: worldPoint.x - this.pos.x,
      y: worldPoint.y - this.pos.y
    };
    this.torque += r.x * force.y - r.y * force.x;
  }

```

On accumule les forces et les couples pendant la frame, puis on les applique dans l'intégrateur. Le bras de levier $\vec{r}$ est la distance entre le centre de masse et le point d'application.

Étape 3 : L'intégrateur

Euler explicite — linéaire + angulaire

```

update(dt) {
  // --- Linéaire ---
  const ax = this.force.x / this.mass;
  const ay = this.force.y / this.mass;
  this.vel.x += ax * dt;
  this.vel.y += ay * dt;
  this.pos.x += this.vel.x * dt;
  this.pos.y += this.vel.y * dt;

  // --- Angulaire ---
  const alpha = this.torque / this.inertia;
  this.angularVelocity += alpha * dt;
  this.angle += this.angularVelocity * dt;

  // --- Reset accumulateurs ---
  this.force.x = 0; this.force.y = 0;
  this.torque = 0;
}

```

Exactement le même schéma qu'Euler en translation — juste **dupliqué** pour la rotation. C'est la beauté de l'analogie linéaire/angulaire.

Étape 4 : Interaction souris

Cliquer pour pousser

```

canvas.addEventListener('click', (e) => {
  const mousePos = getMousePos(e);           // position souris (world)
  const forceDir = {
    x: mousePos.x - body.pos.x,               // direction : vers la souris
    y: mousePos.y - body.pos.y
  };
  const magnitude = 500;                      // force arbitraire
  const len = Math.hypot(forceDir.x, forceDir.y);
  const force = {
    x: (forceDir.x / len) * magnitude,
    y: (forceDir.y / len) * magnitude
  };
});

```

```

    };
    body.applyForce(force, mousePos);           // applique au point cliqué
  });

```

Le point d'application est la **position de la souris** — si on clique loin du centre, on crée un **couple** et le rectangle tourne. Si on clique près du centre, il se déplace sans tourner. C'est l'équivalent du `impactHeight` de la démo billard.

Étape 5 : Rendu

Dessiner le rectangle pivoté

```

function draw() {
  ctx.save();
  ctx.translate(body.pos.x, body.pos.y); // aller au centre
  ctx.rotate(body.angle);                // pivoter
  ctx.fillRect(-w/2, -h/2, w, h);        // dessiner centré
  ctx.restore();
}

```

En Canvas 2D, on utilise `ctx.translate` + `ctx.rotate` pour positionner et orienter le rectangle. L'ordre est important : **d'abord** traduire, **ensuite** pivoter.

Partie 10 : Récapitulatif — Le Corps Rigide Complet

Phase	Linéaire	Angulaire	Code
Accumuler	$\vec{F} + = \vec{F}_{\text{appliquée}}$	$\tau + = r \times F$	<code>applyForce()</code>
Intégrer	$\vec{v} + = (\vec{F}/m) \cdot dt$	$\vec{\omega} + = (\tau/I) \cdot dt$	<code>update()</code>
Déplacer	$\vec{r} + = \vec{v} \cdot dt$	$\theta + = \omega \cdot dt / q$	<code>update()</code>
Reset	$\vec{F} = 0$	$\tau = 0$	<code>update()</code> fin

Table 2: Le cycle complet d'un corps rigide. Chaque frame : accumuler les forces et couples, intégrer (Euler), mettre à jour position et orientation, réinitialiser les accumulateurs.

🗨 Ce qu'on retient

1. Un corps rigide a **deux** dynamiques — linéaire et angulaire — parfaitement **analogues**.
2. Le couple $\tau = r \times F$ dépend du **point d'application** — pas seulement de la force.
3. Le moment d'inertie I dépend de la **répartition de masse** — pas seulement de la masse totale.
4. La vitesse d'un point $\vec{v}_P = \vec{v}_G + (\vec{\omega} \times \vec{r})$ est la clé du glissement vs roulement.
5. La friction de glissement **converge** vers le roulement en ralentissant v_G et en accélérant ω simultanément.
6. L'intégration de la rotation utilise des **quaternions** en 3D (pas de gimbal lock) et un **angle scalaire** en 2D.